

ЗАВРШНИ РАД

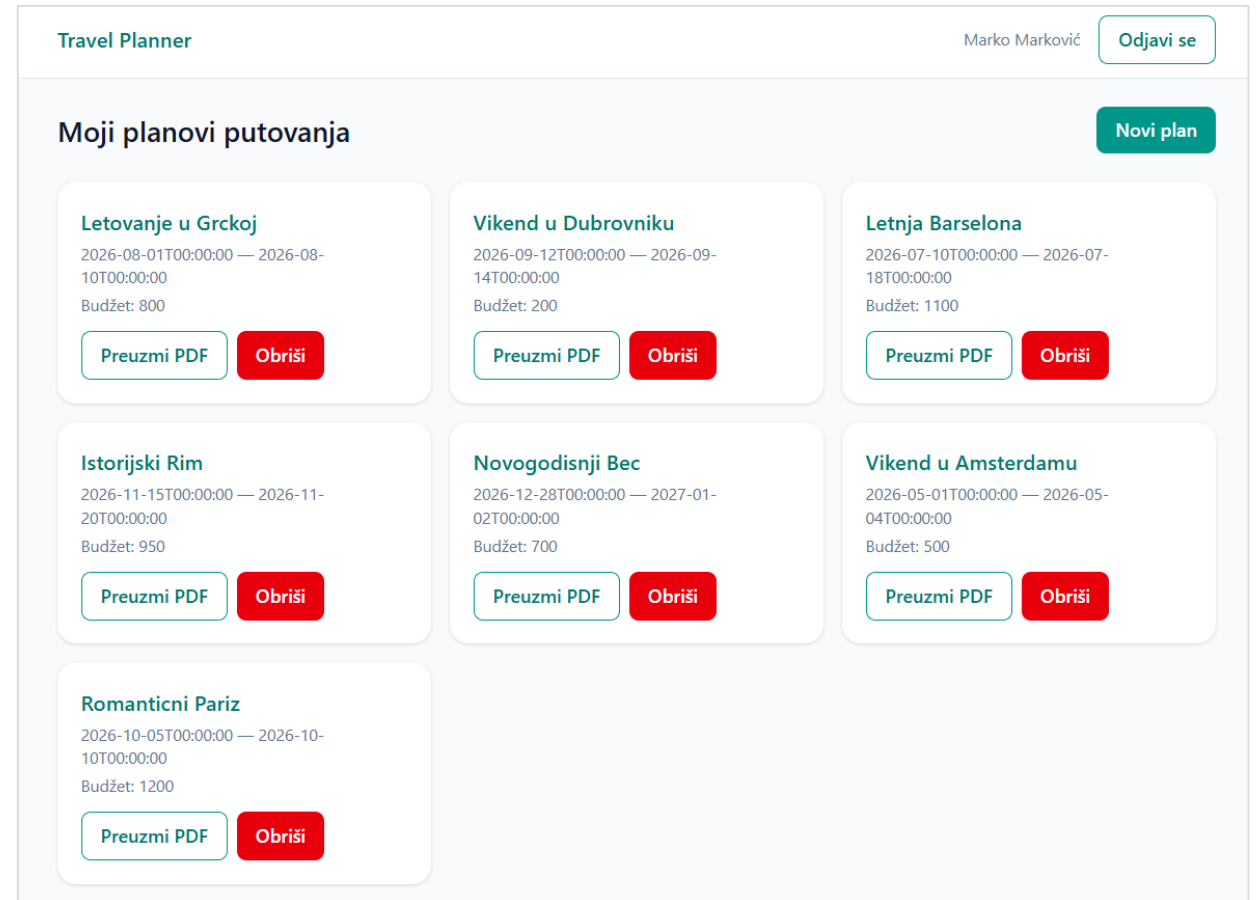
Имплементација обсервабилности микросервисне архитектуре на примеру TravelPlanner апликације

Дајана Булић

Ментор: др Себастијан Стоја, доцент

Travel Planner — постојећа апликација

- Микросервисна веб апликација за планирање путовања
- Планови путовања, дестинације, активности, буџет, checklist
- Дељење плана путем кода или QR кода са другим особама
- Реализована кроз четири одвојена сервиса на Microsoft Service Fabric платформи



Микросервиси су тешки за праћење

Један кориснички захтев пролази кроз више процеса, база и мрежних позива — монолитна интуиција „погледај један логфајл” више не важи.

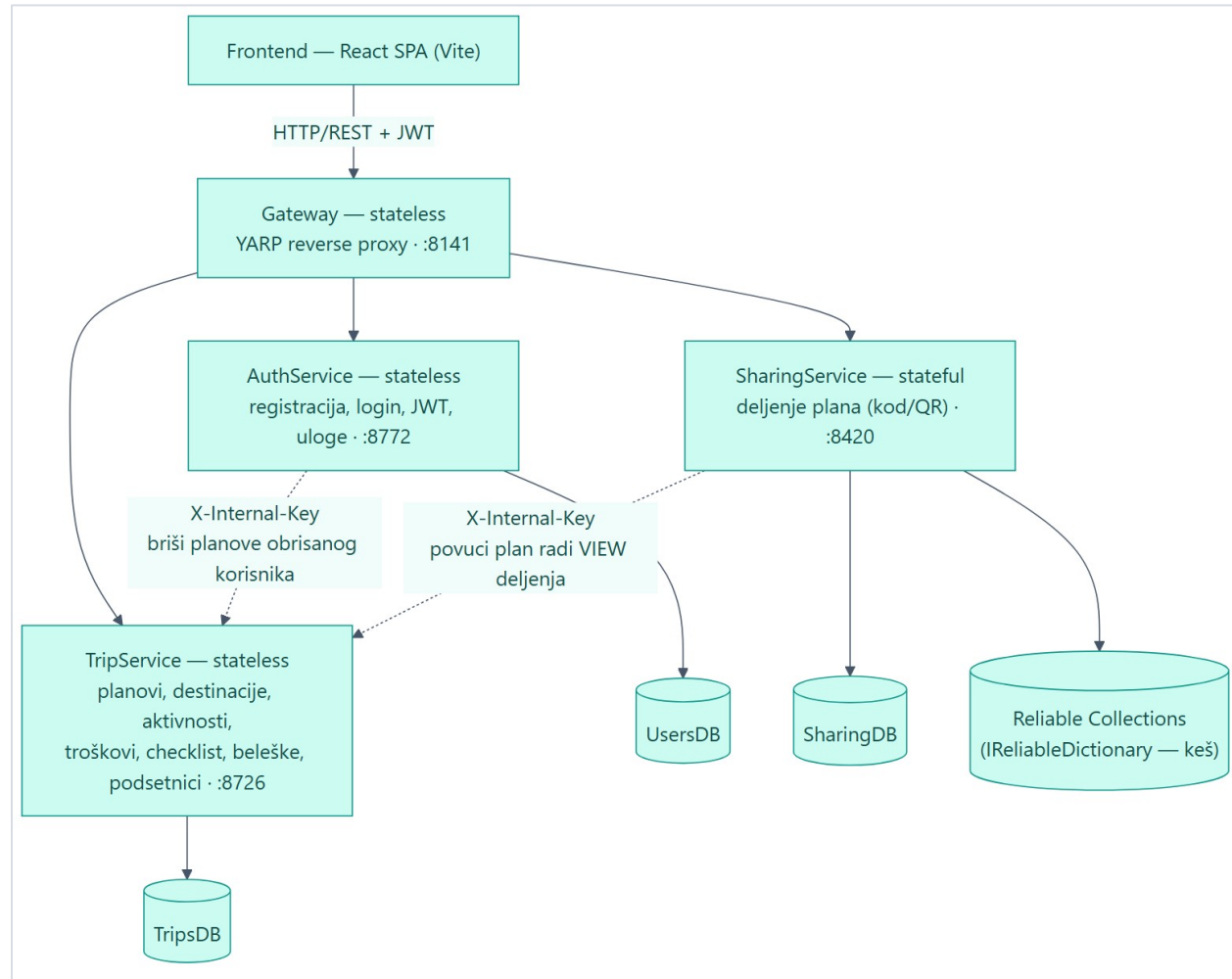
- „Зашто је овај захтев спор?”
- „Где тачно долази до грешке?”
- „Да ли је систем тренутно здрав?”

Питања на која постојећи Travel Planner систем није могао да одговори.

Шта овај рад доказује

- Анализирати постојећи систем са аспекта обсервабилности и идентификовати конкретне недостатке
- Надоградити систем слојем обсервабилности заснованим на индустријском стандарду OpenTelemetry
- Не мењати постојећу пословну логику (планове, дестинације, трошкове)
- Доказати надоградњу кроз стварно извршавање, аутоматизоване тестове и мерљиве резултате — не само инсталацију алата

Архитектура пре надоградње (AS-IS)



- Gateway, AuthService, TripService — stateless
- SharingService — stateful (Reliable Collections кеш)
- База по сервису (database-per-service)

Идентификовани недостаци

- Немогуће праћење захтева кроз више сервиса — нема correlation/trace ID
- Одсуство метрика перформанси — CPU, меморија, латенција, стопа грешака
- „Тихе“ грешке — примери у коду који гутају грешке без иједног трага у логу
- Одсуство health check механизма — ниједан сервис не пријављује да ли ради
- Нестандардизовано, минимално логовање — 0 позива ILogger пре овог рада

OpenTelemetry + Grafana stack

Један, vendor-неутралан стандард уместо низа ad-hoc закрпа.

OpenTelemetry

Заједнички API/SDK за traces, metrics, logs — auto-инструментација за ASP.NET Core, HttpClient, SqlConnection

OpenTelemetry Collector

Централна тачка прикупљања телеметрије (OTLP, gRPC) — раздваја емисију од складиштења

Prometheus

Складиштење и скрапе метрика перформанси и извршног окружења

Grafana Tempo

Дистрибуирано праћење захтева (distributed tracing)

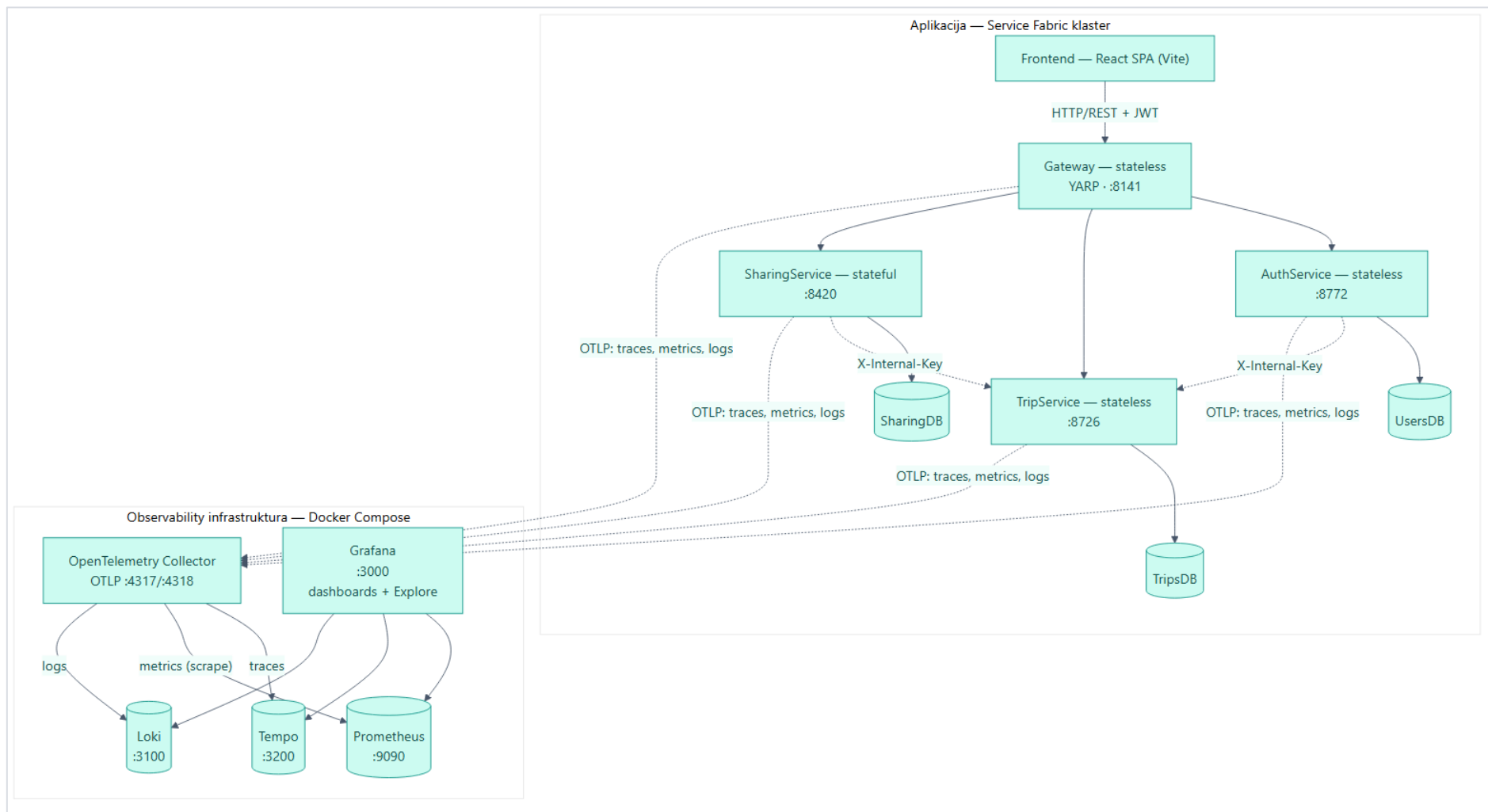
Grafana Loki

Централизовани, структурисани логови

Grafana

Јединствен визуелни слој — унакрсна навигација
log → trace → metric

TO-BE архитектура — додат observability слој

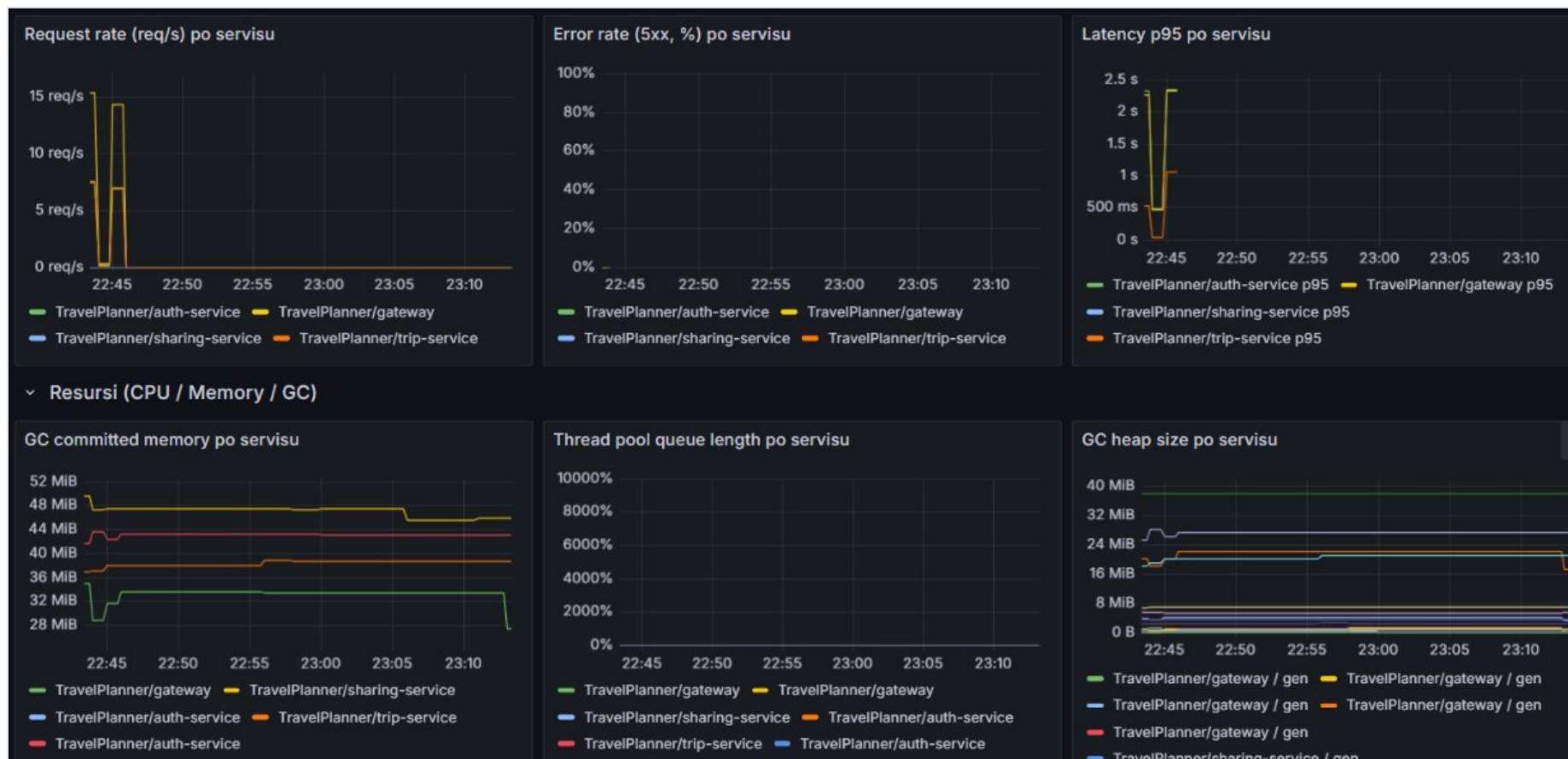


Health checks

- /health/live — да ли процес живи (одлуке о рестарту)
- /health/ready — да ли су критичне зависности спремне
- Gateway проверава мрежну доступност downstream сервиса
- Остали сервиси проверавају доступност сопствене SQL базе
- Одговор: структурисан JSON, читљива порука грешке

```
{
  "status": "Healthy",
  "totalDurationMs": 179.8175,
  "checks": [
    {
      "name": "auth-service",
      "status": "Healthy",
      "durationMs": 159.1745,
      "description": "http://localhost:8772 je dostupan.",
      "error": null
    },
    {
      "name": "trip-service",
      "status": "Healthy",
      "durationMs": 91.5194,
      "description": "http://localhost:8726 je dostupan.",
      "error": null
    },
    {
      "name": "sharing-service",
      "status": "Healthy",
      "durationMs": 92.2722,
      "description": "http://localhost:8420 je dostupan.",
      "error": null
    }
  ]
}
```

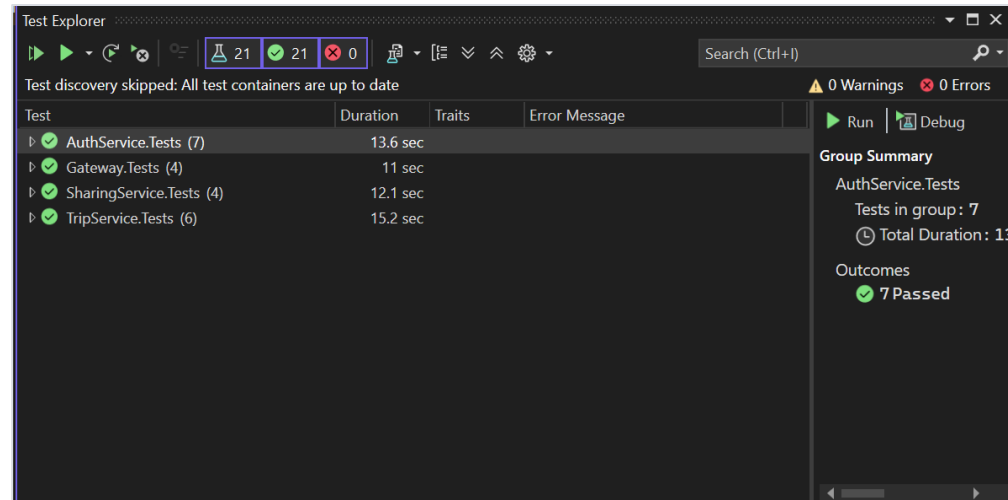
Grafana dashboard — све на једном месту



RED метрике, ресурси (CPU/меморија/GC) и ток логова — филтрирано по сервису, на једном dashboard-у.

Тестирање — стратегија

- Четири xUnit пројекта — по један за сваки сервис, прав Kestrel host, без mock-а базе
- HealthEndpointsTests — интеграциони тестови health check endpoint-a
- ObservabilityPipelineTests — да ли се OTel инструментација исправно активира
- TripClientTests — регресиони тест за поправљен „тихи“ catch блок



Доказ да тестови стварно нешто проверавају

Намерно враћен „тихи” catch блок (уклоњен LogWarning позив) — резултат: тачно 1 тест пада, остали пролазе.

The screenshot shows the Visual Studio Test Explorer interface. At the top, a summary bar indicates 'Test run finished: 3 Tests (2 Passed, 1 Failed, 0 Skipped) run in 710 ms'. Below this, a table lists the tests and their results:

Test	Duration	Traits	Error Message
AuthService.Tests (7)	13.6 sec		
AuthService.Tests (7)	13.6 sec		
HealthEndpointsTests (3)	7.9 sec		
ObservabilityPipelineTests (1)	5.3 sec		
TripClientTests (3)	409 ms		
DeleteUserTripsAsync_WhenSuc...	2 ms		
DeleteUserTripsAsync_WhenTri...	397 ms		Assert.Con...
DeleteUserTripsAsync_WhenTri...	10 ms		
Gateway.Tests (4)	11 sec		
SharingService.Tests (4)	12.1 sec		
TripService.Tests (6)	15.2 sec		

The right pane shows the 'Test Detail Summary' for the failed test: 'AuthService.Tests.TripClientTests.DeleteUserTripsAsync_WhenTripServiceReturnsError_ReturnsFalseAndLogsWarn'. The source is 'TripClientTests.cs line 46' and the duration is '397 ms'. The message is 'Assert.Contains() Failure: Filter not matched in collection Collection: []'. The stack trace shows the test method name and ends with '--- End of stack trace from previous location ---'.

Демонстрациони сценарији над стварним системом

Сценарио 1 — нормалан захтев

Trace кроз три сервиса, 9 SQL child span-ова, логови повезани преко trace_id

Сценарио 2 — грешка

Health check аутоматски пријављује Unhealthy, опоравак за ~7s

Сценарио 3 — оптерећење

k6: 780 захтева, p95 клијент/сервер латенција 2.12s / 2.14s

Сценарио 4 — спора зависност

Trace тачно локализује узрок: 23.9s укупно, SELECT свега 14.5ms

Дистрибуирано праћење захтева у пракси



Сценарио 1: Gateway → SharingService → TripService, са 9 угнежђених SQL упита — све аутоматски, без custom кода.

Шта је постигнуто

- Идентификовано 5 конкретних недостатака постојећег система
- Имплементирана надоградња заснована на OpenTelemetry + Grafana stack-у, без измене пословне логике
- Верификовано на два независна начина: 21 аутоматизован тест + 4 демонстрациона сценарија над стварним системом

Правци даљег развоја

- Формално alerting решење изнад Grafana dashboard-a
- Инструментација frontend-a ради потпуног end-to-end trace-a
- Аутоматизација k6 load теста као дела CI/CD pipeline-a

Хвала на пажњи

Питања?